2026

TwoVago

PROYECTO FINAL: DESARROLLO WEB- ENTREGA 1: DISEÑO INICIAL

HERNANDEZ OCHOA EMANUEL-COBOS VALENCIA JESUS ALFREDO-BAEZ RODRIGUEZ FRANCISCO

LISTI | Docente: José Rafael Rojano Caceres| 10/11/2026

Contenido

Introducción 3

Problemática 4

Justificación 4

Objetivo general 5

Objetivos específicos 5

Actores 5

Alcance de la primera versión 5

Requerimientos y reglas de negocio 6

- Requerimientos funcionales 6
- Requerimientos no funcionales 6
- Reglas de negocio 6

Arquitectura del sistema: tres capas 7

Modelo de datos 8

Entidades y datos principales 9

Estados previstos 9

Integridad y disponibilidad 9

Diseño de API REST 9

- Autenticación y usuarios – Francisco 10
- Hoteles – Jesús 10
- Habitaciones – Jesús 10
- Reservaciones, pagos y respuestas HTTP 10
- Reservaciones – Emanuel – Francisco 10
- Pagos simulados – Emanuel 11
- Códigos HTTP previstos 11

Flujo de reservaciones y pagos simulados 12

- Reglas del flujo 13

Diseño de interfaces y navegación 13

- Navegación del cliente 13

Navegación administrativa 14
Diseño adaptable y accesibilidad 14
Tecnologías y responsabilidades 14
Tecnologías previstas 14
Distribución de responsabilidades 14
Estado del proyecto y próximos pasos 15
Estado al corte de esta versión 15
Pendientes para completar el proyecto 15
Enlace del proyecto 15
Evidencias 16
1. Historial de commits en GitHub 16
2. Documento de análisis y diseño 17
3. Modelo de datos 17
4. Historial local de Git en WSL 18
Evidencias para versiones posteriores 18

Introducción

La plataforma se llamará TwoVago con búsqueda y reservación de hoteles que nos permite consultar establecimientos registrados en una base de datos, comparar sus características y precios, revisar la disponibilidad de habitaciones y gestionar las reservaciones. Los usuarios crean una cuenta y consultan con su perfil revisando así sus reservaciones actuales, el historial de las estancias y sus cancelaciones, así mismo, el sistema cuenta con un plazo de 10 minutos para confirmar cada reservación y pagar. También contemplamos un módulo de administrar los registros, actualizar hoteles y habitaciones, consultando así las reservaciones, su comunicación será mediante una API REST e intercambiará mediante el formato JSON.

Problemática

La búsqueda de alojamiento implica revisar siempre varias opciones de hospedaje, comparar precios y ver si está disponible ciertas habitaciones para distintas fechas y verificar la cantidad de huéspedes que habrá. Además, una vez iniciada una reservación, se necesita conocer si permanece el pago pendiente, si ya fue confirmada, cancelada o expiró por no confirmar. Cuando se consulta por separado, el usuario pierde continuidad con la búsqueda, la selección de una habitación y su reservación, por lo que TwoVago propone reunir estas opciones en una misma plataforma.

Justificación

el sistema permitirá concentrar en un solo sitio la consulta de los hoteles, comparar las habitaciones y gestionar las reservaciones, este registro consultará las reservaciones vigentes, estancias concluidas, cancelaciones y pagos simulados. Desde nuestro punto de vista, este proyecto integra interface web, servicios HTTP con Fetch, diseño de una API REST, Hibernate/JPA y almacenamiento en MySQL, por ello la propuesta se desarrollará con Java y Spring Boot en el backend y HTML, CSS y JavaScript en el frontend.

Objetivo general

Desarrollar una aplicación web que permita buscar hoteles, consultar habitaciones y gestionar reservaciones mediante una arquitectura de 3 capas, usando Java con Spring Boot, MySQL, Hibernate/JPA, HTML, CSS y JavaScript.

Objetivos específicos

- Organizar un catálogo de hoteles y habitaciones almacenados en una base de datos.
- Permitir búsquedas por destino, fechas, cantidad de huéspedes y criterio de precio o categoría.
- Administrar cuentas de clientes y permisos de administración.
- Registrar reservaciones y validar la disponibilidad de las habitaciones.
- Incorporar un pago simulado que tenga un límite de 10 minutos para confirmar la reservación.
- Conservar los estados de las reservaciones para consultar sus operaciones actuales, estancias completadas, cancelaciones y expiraciones.

Actores

Cliente: Registrarse e iniciar sesión, buscar hoteles, consultar habitaciones, crear reservaciones, confirmar pagos simulados, consultar perfil, reservaciones, historial y cancelaciones.

Administrador: Registrar y modificar hoteles, administrar habitaciones, desactivar registros cuando corresponda y consultar las reservaciones del sistema.

Alcance de la primera versión

La plataforma contará con un catálogo de hoteles de demostración almacenados en una base de datos, esta incluirá usuarios, hoteles, habitaciones, reservaciones y pagos simulados. No se integrarán servicios externos de reservación ni pagos reales, por lo que los pagos serán ficticios y servirán para validar el flujo de confirmación y vencimiento de la reservación.

Requerimientos y reglas de negocio

Requerimientos funcionales

- Registrar usuarios.
- Iniciar y cerrar sesión.
- Consultar los hoteles registrados.
- Buscar hoteles por ciudad, fechas y número de huéspedes.
- Mostrar información, fotografías y servicios de cada hotel.
- Consultar habitaciones disponibles.
- Crear reservaciones.
- Generar un folio para cada reservación.
- Implementar un pago simulado con límite de 10 minutos.
- Confirmar reservaciones cuando el pago se complete dentro del plazo.
- Expirar las reservaciones que no se paguen a tiempo.
- Cancelar reservaciones confirmadas.
- Consultar reservaciones pendientes y confirmadas.
- Consultar el historial de estancias completadas.
- Consultar reservaciones canceladas y expiradas.
- Permitir al administrador gestionar hoteles y habitaciones.

Requerimientos no funcionales

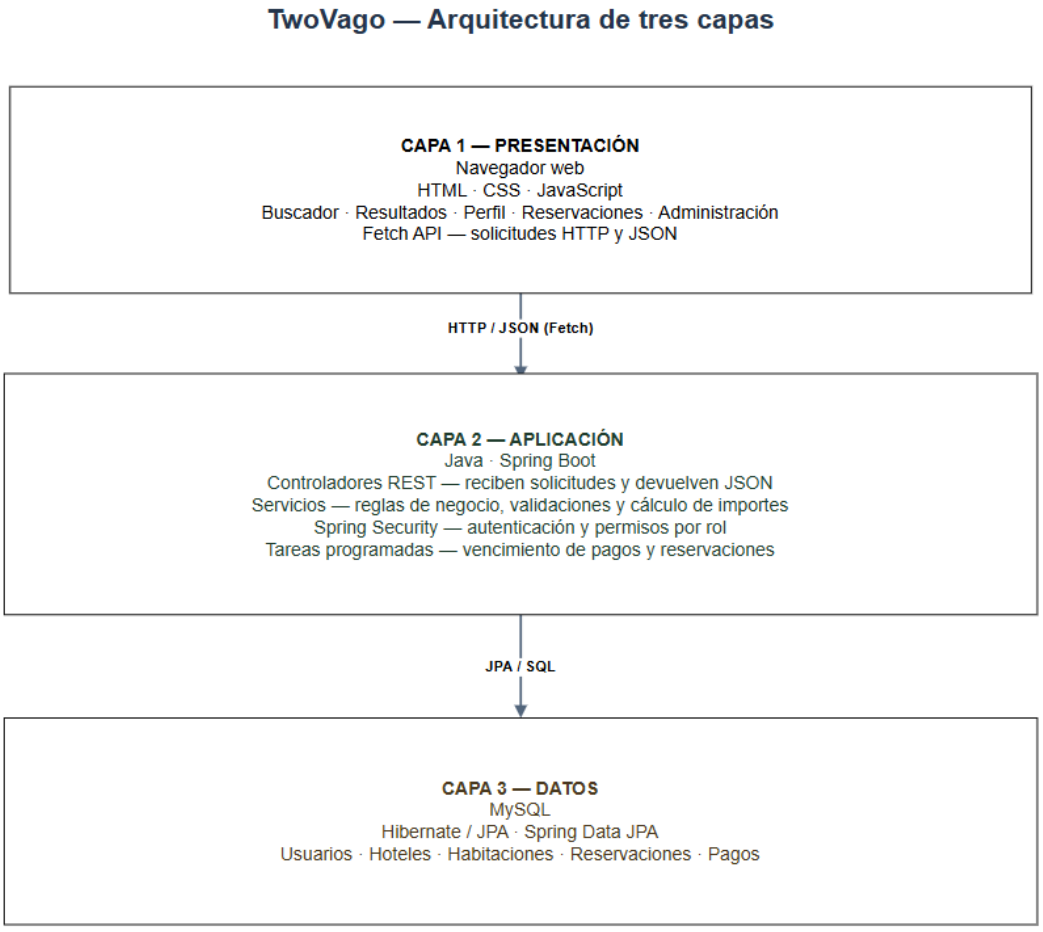
- Backend con Java y Spring Boot; frontend con HTML, CSS y JavaScript.
- Persistencia en MySQL mediante Hibernate/JPA y Spring Data JPA.
- Comunicación mediante API REST, Fetch, HTTP y JSON.
- Contraseñas almacenadas como hash; no se devolverán en respuestas.
- Validación de datos y control de acceso por roles CLIENTE y ADMIN.
- Validación de disponibilidad para impedir reservaciones activas con fechas superpuestas.
- Desarrollo y ejecución desde WSL; código fuente administrado en GitHub.

Reglas de negocio

- Para reservar, el usuario debe iniciar sesión.
- La fecha de salida debe ser posterior a la fecha de entrada.
- La habitación debe tener capacidad suficiente para los huéspedes.
- Una habitación no puede tener reservaciones bloqueantes con fechas superpuestas.
- Cada reservación nueva genera un plazo de pago de 10 minutos.
- Si el plazo vence, la reservación pasa a EXPIRADA y deja de bloquear la habitación.
- Una reservación CONFIRMADA puede cancelarse antes de la fecha de entrada.
- Los pagos son simulados; no se realizan transacciones bancarias reales.

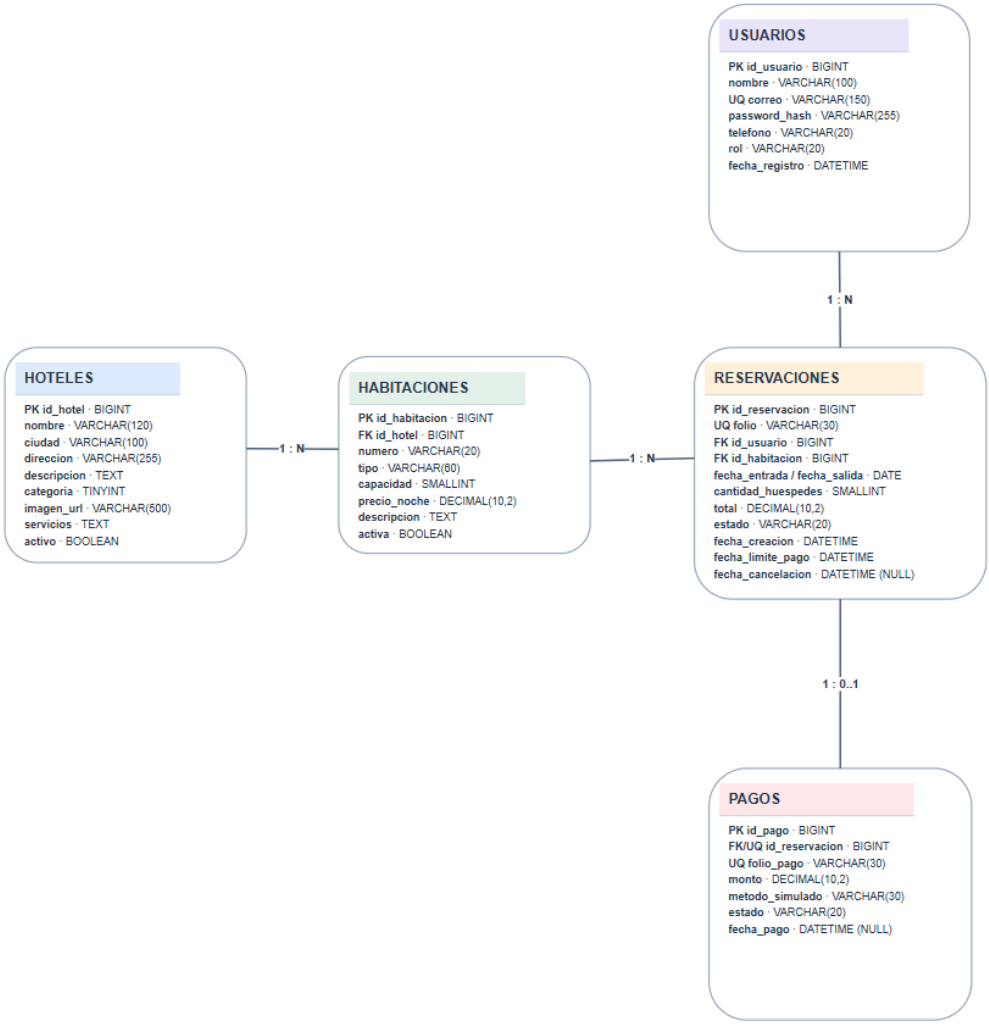
Arquitectura del sistema: tres capas

La arquitectura separa a la presentación, la lógica y el almacenamiento, el navegador envía solicitudes a la API REST de Spring Boot, los servicios aplican las reglas de negocio y acceden a MySQL mediante Hibernate/JPA y Spring Data JPA.



Modelo de datos

El modelo inicial tiene cinco entidades, las relaciones nos permiten asociar hoteles con sus habitaciones, Usuarios con sus reservaciones y reservaciones con sus pagos simulados.



Relación	Cardinalidad	Significado
HOTELES – HABITACIONES	1:N	Un hotel puede tener varias habitaciones; cada habitación pertenece a un solo hotel.
HABITACIONES – RESERVACIONES	1:N	Una habitación puede aparecer en distintas reservaciones a lo largo del tiempo; cada reservación corresponde a una sola habitación.
USUARIOS – RESERVACIONES	1:N	Un usuario puede realizar varias reservaciones; cada reservación pertenece a un solo usuario.
RESERVACIONES – PAGOS	1:0..1	Una reservación puede tener cero o un registro de pago; cada pago corresponde a una sola reservación.

Entidades y datos principales

Entidad	Campos
USUSARIOS	id_usuario (PK), nombre, correo (único), password_hash, telefono, rol, fecha_registro.
HOTELES	id_hotel (PK), nombre, ciudad, direccion, descripcion, categoria, imagen_url, servicios, activo.
HABITACIONES	id_habitacion (PK), id_hotel (FK), numero, tipo, capacidad, precio_noche, descripcion, activa. Restricción única: (id_hotel, numero).
RESERVACIONES	id_reservacion (PK), folio (único), id_usuario (FK), id_habitacion (FK), fechas, cantidad_huespedes, total, estado, fecha_creacion, fecha_limite_pago y fecha_cancelacion.
PAGOS	id_pago (PK), id_reservacion (FK y único), folio_pago (único), monto, metodo_simulado, estado y fecha_pago.

Estados previstos

Reservación: PENDIENTE_PAGO, CONFIRMADA, CANCELADA, EXPIRADA, COMPLETADA.

Pago: PENDIENTE, APROBADO, RECHAZADO, EXPIRADO.

Integridad y disponibilidad

El diseño define correo y folios únicos, número de habitación único dentro de cada hotel y su relación con sólo un pago por reservación, su disponibilidad se considera bloquear las reservaciones CONFIRMADA y PENDIENTE_PAGO cuyo plazo siga vigente. Las reservaciones CANCELADA y EXPIRADA dejan de bloquear fechas y la validación con creación deberán ejecutarse con transacción para controlar muchas solicitudes. El historial de estancias, cancelaciones y expiraciones se obtienen de los estados de la tabla RESERVACIONES, no se proponen tablas para estos apartados.

Diseño de API REST

La API REST conectará el frontend con el backend mediante HTTP y JSON. La URL base prevista para desarrollo es http://localhost:8080/api. La autenticación se plantea mediante sesiones administradas por Spring Security.

Autenticación y usuarios – Francisco

Metodo	Endpoint	Función/ acceso
POST	/api/auth/registro	Registrar cliente - Público
POST	/api/auth/login	Iniciar sesión - Público
POST	/api/auth/logout	Cerrar sesión - Autenticado
GET	/api/usuarios/perfil	Consultar perfil propio - Autenticado
PUT	/api/usuarios/perfil	Actualizar datos personales - Autenticado

Hoteles – Jesús

Metodo	Endpoint	Función/ acceso
GET	/api/hoteles	Listar y buscar hoteles - Público
GET	/api/hoteles/{id}	Consultar hotel - Público
POST	/api/hoteles	Registrar hotel - ADMIN
PUT	/api/hoteles/{id}	Actualizar hotel - ADMIN
DELETE	/api/hoteles/{id}	Desactivar hotel - ADMIN

Habitaciones – Jesús

Metodo	Endpoint	Función/ acceso
GET	/api/habitaciones	Consultar habitaciones; admite hotelId - Público
GET	/api/habitaciones/{id}	Consultar habitación - Público
GET	/api/habitaciones/disponibles	Consultar disponibilidad - Público
POST	/api/habitaciones	Registrar habitación - ADMIN
PUT	/api/habitaciones/{id}	Actualizar habitación - ADMIN
DELETE	/api/habitaciones/{id}	Desactivar habitación - ADMIN

Reservaciones, pagos y respuestas HTTP

Reservaciones – Emanuel – Francisco

Metodo	Endpoint	Función/ acceso
POST	/api/reservaciones	Crear reservación - CLIENTE
GET	/api/reservaciones/{id}	Consultar reservación propia - Propietario o ADMIN
GET	/api/reservaciones/mis-reservas	Consultar pendientes y confirmadas - CLIENTE
GET	/api/reservaciones/historial	Consultar estancias completadas - CLIENTE
GET	/api/reservaciones/cancelaciones	Consultar cancelaciones - CLIENTE
GET	/api/reservaciones/expiradas	Consultar reservaciones vencidas - CLIENTE
PUT	/api/reservaciones/{id}/cancelar	Cancelar reservación confirmada - Propietario o ADMIN
GET	/api/admin/reservaciones	Consultar todas las reservaciones ADMIN

Pagos simulados – Emanuel

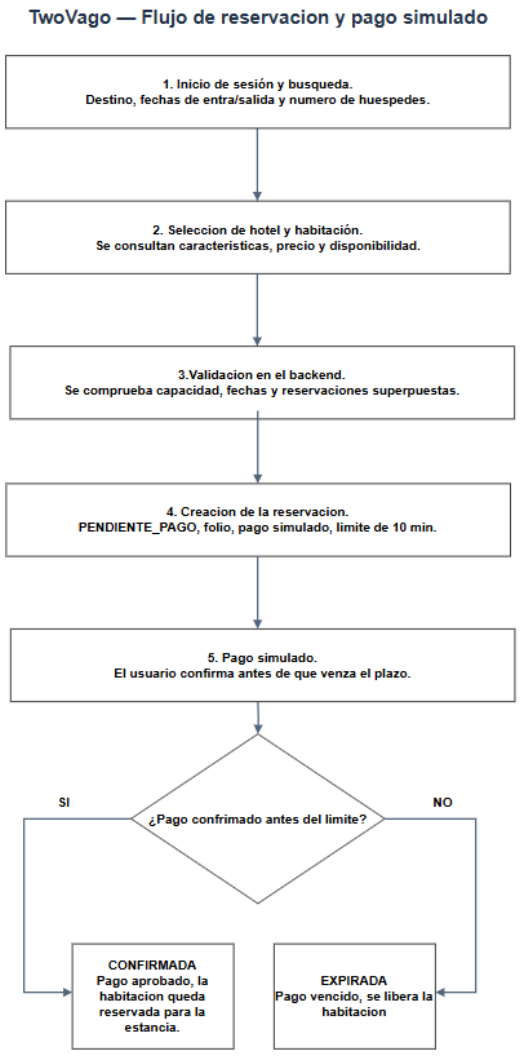
Metodo	Endpoint	Función/ acceso
GET	/api/pagos/{id}	Consultar pago propio - Propietario o ADMIN
POST	/api/pagos/{id}/confirmar	Confirmar pago simulado - Propietario

Códigos HTTP previstos

Código	Uso previsto
200	Consulta o acción exitosa.
201	Creación de usuario, hotel, habitación o reservación.
400	Datos inválidos o fechas incorrectas.
401 / 403	Falta de sesión o permisos insuficientes.
404	Recurso no encontrado.
409	Conflicto de disponibilidad, pago vencido o transición no permitida.
500	Error no previsto en el servidor.

Flujo de reservaciones y pagos simulados

El proceso inicia cuando un usuario autenticado busca un hotel y selecciona una habitación. El backend vuelve a validar la disponibilidad, calcula el total y crea una reservación en estado PENDIENTE_PAGO, junto con un folio y un pago simulado. El usuario dispone de 10 minutos para confirmar.



Reglas del flujo

- El total se calcula como número de noches × precio por noche y se guarda al crear la reservación.
- El plazo de pago se registra en fecha_limite_pago; el temporizador del frontend es informativo y el servidor decide si el pago está dentro del plazo.
- Si el pago se confirma a tiempo, el pago pasa a APROBADO y la reservación a CONFIRMADA.
- Si vence el plazo, el pago pasa a EXPIRADO y la reservación a EXPIRADA; la habitación deja de estar bloqueada.
- Una reservación confirmada puede cancelarse antes de la fecha de entrada.
- El historial muestra estancias COMPLETADAS; cancelaciones y expiraciones se consultan en apartados separados.

Diseño de interfaces y navegación

La implementación visual se realiza durante el desarrollo del frontend ya que se dispone de lo expuesto y la relación con los endpoints.

Pantalla	Contenido y acciones
Inicio	Logotipo, navegación, registro/inicio de sesión y buscador por destino, fechas y huéspedes.
Resultados	Listado de hoteles con ubicación, categoría, servicios, precio y filtros.
Detalle del hotel	Descripción, imágenes, servicios, habitaciones, capacidad y precio por noche.
Registro / inicio de sesión	Datos de cuenta y acceso del usuario.
Resumen de reservación	Hotel, habitación, fechas, huéspedes, precio por noche, noches e importe total.
Pago simulado	Folio, importe, método ficticio y contador de 10 minutos.
Confirmación	Folio, hotel, habitación, fechas, importe y estado confirmado.
Perfil e historial	Datos personales, reservaciones actuales, estancias, cancelaciones y expiraciones.
Panel administrativo	Gestión de hoteles, habitaciones y consulta de reservaciones

Navegación del cliente

Inicio → búsqueda → resultados → detalle del hotel → selección de habitación → registro o inicio de sesión → resumen → pago simulado → confirmación. Si el plazo termina, la reservación pasa a EXPIRADA y la habitación se libera.

Navegación administrativa

Inicio de sesión → panel administrativo → gestión de hoteles → gestión de habitaciones → consulta de reservaciones.

Diseño adaptable y accesibilidad

- Adaptación a computadoras, tabletas y teléfonos.
- Menús sencillos, campos identificados y botones con etiquetas claras.
- Contraste suficiente, mensajes de error comprensibles e indicadores de carga.
- Confirmación antes de cancelar reservaciones.

Tecnologías y responsabilidades

Tecnologías previstas

Componente	Tecnología	Uso
Frontend	HTML, CSS y JavaScript	Estructura, estilos e interacción de las pantallas.
Comunicación	Fetch API, HTTP y JSON	Consumo de los endpoints REST.
Backend	Java y Spring Boot	Controladores, servicios, validaciones y reglas de negocio.
Persistencia	Hibernate/JPA y Spring Data JPA	Mapeo de entidades y acceso a datos.
Base de datos	MySQL	Almacenamiento de usuarios, hoteles, habitaciones, reservaciones y pagos.
Entorno	WSL, Visual Studio Code y Git	Desarrollo, terminal, control de versiones y sincronización con GitHub.

Distribución de responsabilidades

Integrante	Responsabilidades
Jesús Alfredo Cobos Valencia	Coordinación e integración del repositorio; análisis y documentación; módulos de hoteles y habitaciones; búsqueda y filtros; consistencia entre modelo, endpoints e interfaces
Emanuel Hernández Ochoa	Módulo de reservaciones y pagos simulados; validación de fechas, capacidad y disponibilidad; cálculo de importes; folios; límite de 10 minutos; expiración, cancelaciones y pruebas del módulo
Francisco Baez Rodríguez	Registro, autenticación, autorización y perfil; vistas de cuenta; reservaciones e historial en coordinación con Emanuel; navegación y pruebas de permisos y privacidad.

Estado del proyecto y próximos pasos

Estado al corte de esta versión

- Existe un repositorio GitHub llamado desarrollo-web con carpetas backend, frontend, database y documentación.
- Se encuentran documentados el análisis, el modelo de datos, el diagrama ER, los endpoints REST, el diseño de interfaces y el flujo de reservaciones.
- Se definió una arquitectura de tres capas y se distribuyeron responsabilidades por módulos.
- El backend, el frontend funcional y la base de datos física todavía no se han implementado.
- No se reportan pruebas funcionales de API en este hito porque aún no existe backend ejecutable.

Pendientes para completar el proyecto

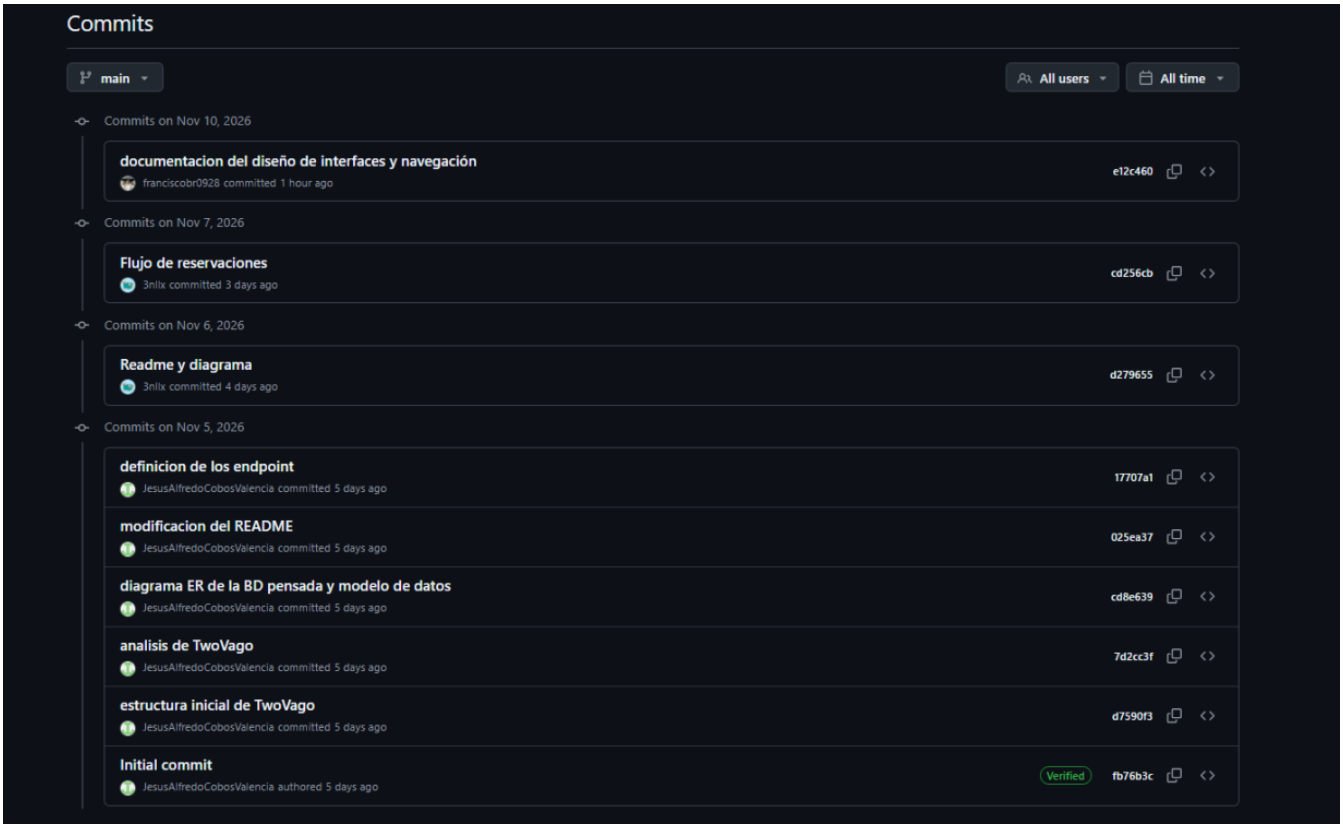
- Completar la documentación del Entregable 1 y validar los archivos que agregará Francisco.
- Verificar que cada integrante tenga contribuciones identificables en GitHub y que los registros individuales de IA reflejen su uso real.
- Publicar el Release 1 una vez revisadas las evidencias y los commits.
- Implementar el backend Spring Boot y la conexión con MySQL.
- Desarrollar las interfaces y conectarlas con la API mediante Fetch.
- Ejecutar pruebas de cada módulo y pruebas de integración.
- Preparar un video de YouTube accesible, de máximo 15–20 minutos, con participación de todos los integrantes.

Enlace del proyecto

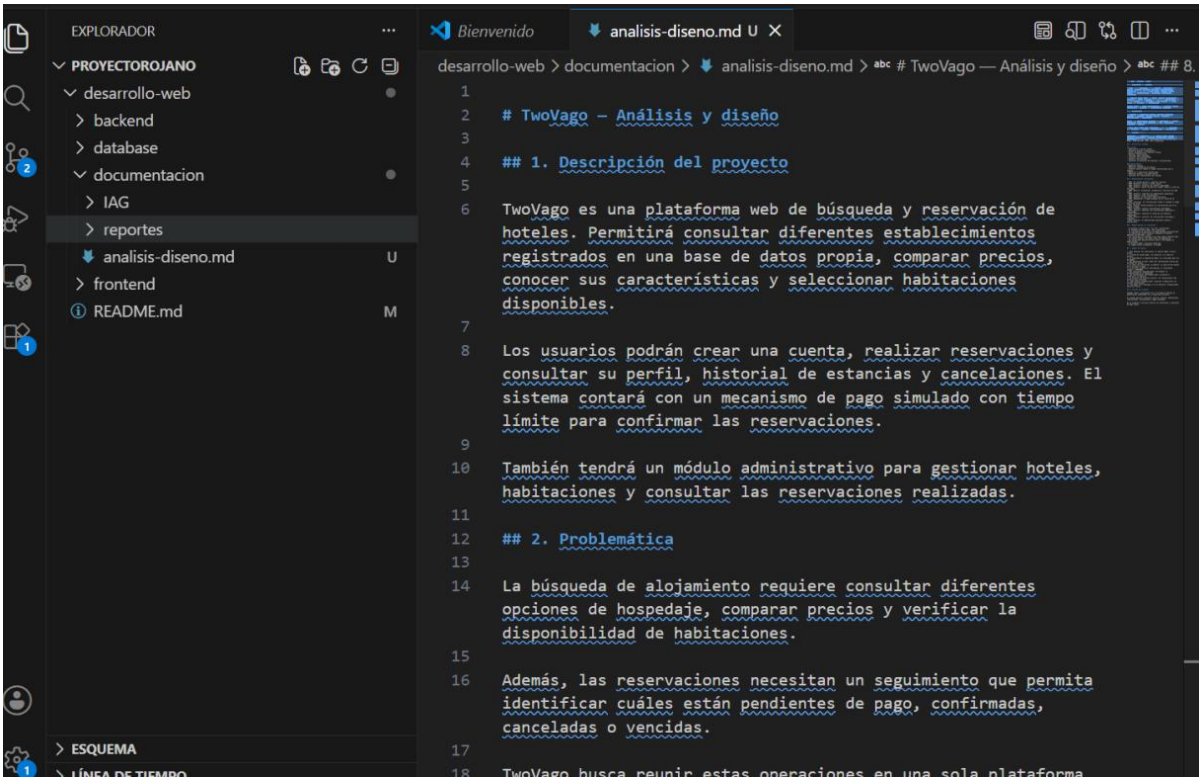
https://github.com/JesusAlfredoCobosValencia/desarrollo-web

Evidencias

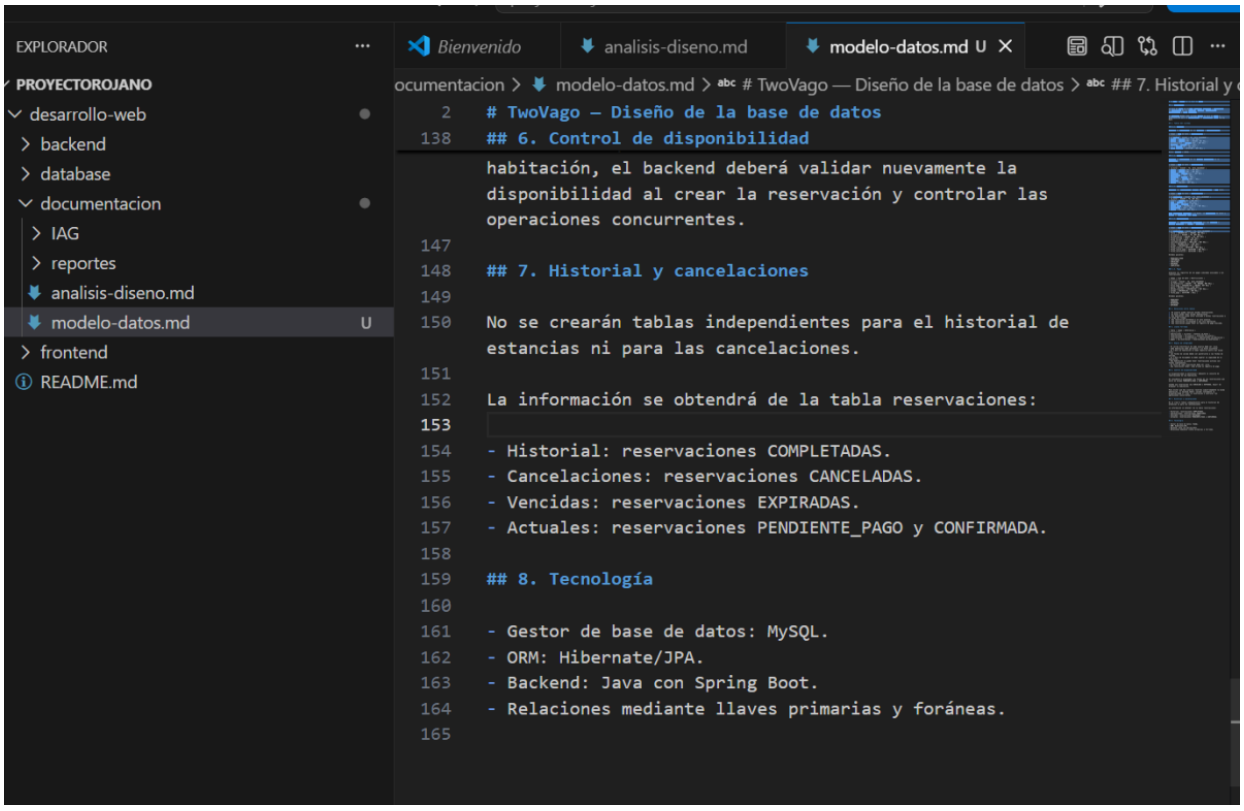
1. Historial de commits en GitHub



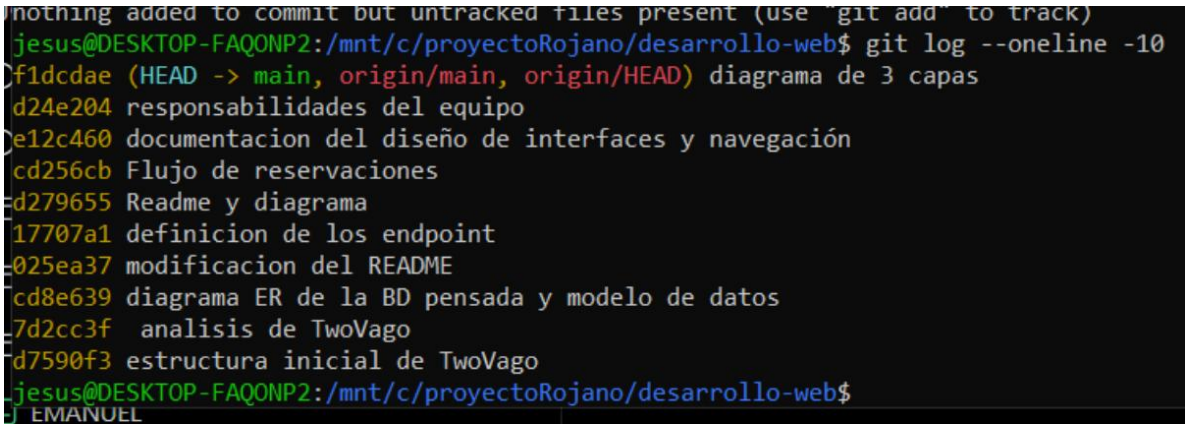
2. Documento de análisis y diseño



3. Modelo de datos



4. Historial local de Git en WSL



Evidencias para versiones posteriores

- Vista del diagrama ER abierto en diagrams.net y del diagrama de arquitectura.
- Capturas de endpoints y diseño de interfaces en VS Code.
- Estructura definitiva de IAG y registros individuales completos.
- Configuración y entrega del webhook, si aplica al repositorio.
- Release R1 publicado en GitHub.
- Ejecución de backend, conexión MySQL, frontend y pruebas funcionales en entregas posteriores.